ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

HPC Lab for CSE

FS 2026
Dr. R. Käppeli,
Dr. F. Gatti, J. Bachmann,
H. Schrader, A. Pietak,
S. Utzinger, V. Vogt,
B. Zemann.

Project 1

Due date: 09 March 2026, 18:00 (Europe/Zurich). Please carefully review the information regarding the deadline on the course webpage. Follow the instructions in Section A.

The first part of this project introduces you to working with Euler cluster and gathering some important performance characteristics. The second part focuses on optimizing general matrix multiplication. Both parts are serial; parallel programming will be covered in forthcoming projects.

You may complete this project in groups of up to five students. In fact, we prefer that you do so. Please clearly indicate your collaborators in the "Discussed with" field of your report. Each student must nevertheless prepare and submit their own independent write-up. Refer to Section A for detailed submission instructions.

1 Euler warm-up [10 points]

Review the Euler cluster documentation[1] to understand its features and usage. In your report, briefly answer the following questions:

1. What is the module system, and how is it used?
2. What is Slurm, and what is its purpose?
3. Write a simple "Hello World" `C`/`C++` program that prints the host name of the machine where it is running.
   Hint: One option is to use the `gethostname` function from the POSIX operating system API declared in `unistd.h`[2].
4. Write a batch script to run your program on a single node. Ensure that the script can target nodes with specific CPUs.
   Hint: See `slurm_job_one.sh` in the provided skeleton code. Slurm has the option `--constraint` to select nodes with specific features. Possible options for selecting nodes with specific CPUs: `EPYC_7742`, `EPYC_7H12`, `EPYC_7763`, `EPYC_9654`.
5. Write another batch script to run your program on two `EPYC_7763` nodes. Verify that the Slurm output file (`slurm-%j.out` by default, where the "`%j`" is replaced by the job ID) contains two different host names.
   Hint: See `slurm_job_two.sh` in the provided skeleton code.

Include the source code, batch scripts, and Slurm output files in your submission (see the instructions in Section A).

2 Performance characteristics [50 points]

In high-performance computing (HPC), understanding the performance limits of computing systems — particularly in relation to the memory system and floating-point operations — is crucial for designing efficient algorithms and optimizing their implementations on specific hardware. The concept of performance characteristics and lightspeed estimates becomes relevant when considering the physical constraints that limit data transfer rates and computation speeds within a system. These constraints are determined not only by the CPU's raw processing power but also by the latency and throughput of the memory hierarchy. Grasping these concepts is essential to appreciate both the theoretical and practical limits of computing, including how data locality and memory bandwidth impact achievable performance. This understanding

[1] https://docs.hpc.ethz.ch/
[2] https://en.wikipedia.org/wiki/Unistd.h

is fundamental for pushing computational boundaries while remaining aware of the practical limitations imposed by physics and current technology.

However, determining the performance characteristics of computing systems is a complex task that requires a multi-faceted approach. Manufacturer documentation provides a baseline of theoretical capabilities and specifications, but real-world performance often deviates from these idealized scenarios due to various factors. Web resources, including insights from HPC centers, provide valuable empirical data and analyses that reflect actual performance characteristics. Another important source of information is direct experimentation through low-level benchmarking. Thus, gaining a comprehensive understanding of computing system performance requires a combination of analyzing manufacturer documentation, leveraging insights from HPC centers and other web resources, and conducting methodical benchmarking experiments.

The goal of this task is to collect performance characteristics and compute lightspeed estimates for a particular architecture. As a refresher (or a concise introduction), we *strongly* encourage you to read the article and appendix [11], as well as Chapters One and Three of the book [3][3].

2.1 Peak performance [16 Points]

In this task, we ask you to compute the core, CPU, node and cluster peak performance for the Euler VII (phase 1 and 2) nodes. Please detail your computation and all sources in your report. In the following, we provide some context and give a step-by-step guide to complete and report the task.

In scientific computing, the focus often lies on floating-point (FP) data, typically utilizing *double precision*. The rate at which the CPU's floating-point unit (FPU) can produce results for multiplication and addition operations is quantified in terms of *floating-point operations per second* (Flops/s, or simply FLOPS). The maximum rate at which a system is capable of executing Flops/s is the so-called (theoretical[4]) *peak (FP) performance*. The peak performance P_{core} of a single core is computed as follows

$$P_{\text{core}} = n_{\text{super}} \times n_{\text{FMA}} \times n_{\text{SIMD}} \times f$$

where

- n_{super} is the superscalarity factor: the number of FP operations the FPU of a core can execute in parallel within a single clock cycle.
- n_{FMA} is the fused multiply-add factor: $n_{\text{FMA}} = 2$ if the FPU supports the FMA instruction (enabling it to perform a multiplication and an addition in a single operation), or $n_{\text{FMA}} = 1$ if the FPU does not support FMA (requiring separate instructions for multiplication and addition).
- n_{SIMD} is the SIMD (Single Instruction, Multiple Data) factor: the number of doubles the FPU can process concurrently with a single SIMD instruction (i.e., the width of the SIMD registers in units of doubles).
- f is the (base) clock frequency (modern multi-core CPUs may dynamically increase their clock frequency to make use of Thermal Design Power (TDP) more efficiently if fewer cores are active).

The peak performance P_{CPU} of a CPU is then

$$P_{\text{CPU}} = n_{\text{cores}} \times P_{\text{core}},$$

where n_{cores} is the number of *physical* cores[5]. The peak performance P_{node} of a node with n_{sockets} identical CPUs is

$$P_{\text{node}} = n_{\text{sockets}} \times P_{\text{CPU}}.$$

Finally, the peak performance P_{cluster} of a cluster consisting of n_{nodes} identical nodes is

$$P_{\text{cluster}} = n_{\text{nodes}} \times P_{\text{node}}.$$

Let's compute the peak performance of the Euler III (2016-2022)[6] nodes as an example. The system comprised 1215 nodes (i.e., $n_{\text{nodes}} = 1215$), each equipped with a single Intel Xeon E3-1585Lv5 CPU (i.e., $n_{\text{sockets}} = 1$). According to the Euler webpage, which conveniently provides detailed specifications,

[3] To access this book, you must be connected to the ETH network, e.g., by using the VPN.

[4] There is a distinction between theoretical and measured peak performance, but we will ignore it for the time being (see here for more information).

[5] As opposed to *logical* cores often present on modern CPUs with *threading* capabilities, such as Hyper-threading (HT) for Intel CPU or Simultaneous Multithreading (SMT) for AMD.

[6] https://docs.hpc.ethz.ch/hardware/cpu_nodes/

each of these CPUs had four cores (i.e., $n_{\text{cores}} = 4$). Additionally, the link to Intel's ARK website[7] offers further technical details, revealing that the CPU is based on the Skylake microarchitecture, operates at a base clock frequency of $f = 3.00$ GHz and supports AVX2 SIMD instructions with 256-bit wide vector registers. This setup allows for processing four 64-bit double-precision FP numbers simultaneously, leading to $n_{\text{SIMD}} = 4$.

The remaining factors to consider for calculating peak performance are the superscalarity and FMA factors. However, these details are not as readily available and typically require a deep dive into technical documents provided by the CPU manufacturer or from technology review sites and academic research. In this specific instance, valuable information is found in Figure 2-9 of Intel's Optimization Reference Manual Volume 1 (around p. 77 in the PDF) which displays the Skylake microarchitecture. This figure indicates that Ports 0 and 1 each can perform one vector FMA operation, establishing the superscalarity factor $n_{\text{super}} = 2$ and the fused multiply-add factor $n_{\text{FMA}} = 2$. Hence, we can compute the (theoretical) peak performance of the Euler III nodes

$$\begin{aligned} P_{\text{core}} &= 2 \times 2 \times 4 \times 3 \text{ GHz} = 48 \text{ GFlops/s}, \\ P_{\text{CPU}} &= 4 \times P_{\text{core}} = 192 \text{ GFlops/s}, \\ P_{\text{node}} &= 1 \times P_{\text{CPU}} = 192 \text{ GFlops/s}, \\ P_{\text{Euler III}} &= 233'280 \text{ GFlops/s} = 233.28 \text{ TFlops/s}. \end{aligned}$$

Your task is now to compute the core, CPU, node and cluster peak performance for the Euler VII (phase 1 and 2) nodes. Detail your computation and all sources like in the above Euler III example. Here are some useful hints for the task:

- Euler VII[8] reveals that each node has two AMD EPYC 7H12 CPUs and provides a link to AMD's general specifications. From AMD's specifications you get that the CPU is part of the AMD EPYC 7002 Series, from which you deduce that it is based on the Zen 2 microarchitecture (codenamed "Rome"). For example, from `https://en.wikipedia.org/wiki/Epyc#History`.

  Likewise, the Euler documentation reveals that each node has two AMD EPYC 7763 CPUs and provides a link to AMD's general specifications. From AMD's specifications you get that the CPU is part of the AMD EPYC 7003 Series, from which you deduce that it is based on the Zen 3 microarchitecture (codenamed "Milan").

- The uops.info[9] website will then provide you with the superscalarity and FMA factors. Alternatively, you could consult AMD's documentation for these factors. However, this approach is slightly more cumbersome. To interpret the table, consider that

  - the vector SIMD register naming convention `https://en.wikipedia.org/wiki/Advanced_Vector_Extensions#Advanced_Vector_Extensions`,
  - throughput (TP) is defined by how many cycles it takes to execute one instruction (i.e., Cycles Per Instruction (CPI)),

In general (beyond this project and course), the following resources are valuable:

- Intel's software and optimization manuals: The easiest way to find them is to search for "Software Developer's Manual" or "Optimization Reference Manual" in your favorite web search engine.
- AMD's software and optimization manuals: The easiest way to find them is to search for "AMD documentation hub" in your favorite web search engine. Once on the hub, search for "Software optimization guide" specific to the CPU model you are interested.
- Intel Intrinsics Guide[10]
- Agner Fog's software optimization resources: `https://www.agner.org/optimize/`
- Latency and throughput information: `https://uops.info/`.
- `https://en.wikichip.org`: Can be a useful resource (despite the ads).

[7] `https://ark.intel.com/content/www/us/en/ark/products/93741/intel-xeon-processor-e3-1585l-v5-8m-cache-3-00-ghz.html`

[8] `https://docs.hpc.ethz.ch/hardware/cpu_nodes/`

[9] `https://uops.info/table.html?search=&cb_lat=on&cb_tp=on&cb_uops=on&cb_ports=on&cb_ZEN2=on&cb_ZEN3=on&cb_measurements=on&cb_doc=on&cb_fma=on`

[10] `https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html`

2.2 Memory Hierarchies [8 Points]

2.2.1 Cache and main memory size

The next task is to identify the parameters of the memory hierarchy on a node of the Euler VII (phase 1 and 2) cluster. Follow the step-by-step guide provided below and detail your results in your project report. Explicitly mention the key differences between the CPUs of phase 1 and 2.

To guide you, we show how this can be achieved for an Euler login node. A useful tool to determine the CPU architecture is the command `lscpu`:

```
[user@eu-login-39 ~]$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                4
On-line CPU(s) list:   0-3
Thread(s) per core:    1
Core(s) per socket:    4
Socket(s):             1
NUMA node(s):          1
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 71
Model name:            Intel(R) Xeon(R) CPU E3-1284L v4 @ 2.90GHz
Stepping:              1
CPU MHz:               3700.048
CPU max MHz:           3800.0000
CPU min MHz:           800.0000
BogoMIPS:              5799.57
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              6144K
L4 cache:              131072K
... TRUNCATED ...
```

From this, we obtain the CPU model (Intel Xeon CPU E3-1284Lv4) featuring four cores (i.e., quad-core) and some basic information on the memory hierarchy (see the highlighted lines). To obtain the total available main memory, one option is the `free` command:

```
[user@eu-login-39 ~]$ free -h --si
               total        used        free      shared  buff/cache   available
Mem:            32G        5.9G        8.5G         81M         17G         25G
Swap:           16G        1.7G         14G
```

Since this is a multi-core CPU, it is valuable to obtain more information on how the memory hierarchy is organized and shared among the cores. One tool for this is provided by `hwloc`. In particular, the following `hwloc-ls` commands will give us the desired information:

```
[user@eu-login-39 ~]$ hwloc-ls --whole-system --no-io --of console
Machine (31GB total) + Package L#0
  NUMANode L#0 (P#0 31GB)
  L4 L#0 (128MB) + L3 L#0 (6144KB)
    L2 L#0 (256KB) + L1d L#0 (32KB) + L1i L#0 (32KB) + Core L#0 + PU L#0 (P#0)
    L2 L#1 (256KB) + L1d L#1 (32KB) + L1i L#1 (32KB) + Core L#1 + PU L#1 (P#1)
    L2 L#2 (256KB) + L1d L#2 (32KB) + L1i L#2 (32KB) + Core L#2 + PU L#2 (P#2)
    L2 L#3 (256KB) + L1d L#3 (32KB) + L1i L#3 (32KB) + Core L#3 + PU L#3 (P#3)
```

From the output, we conclude that each of the four cores has its own L1 instruction (L1i) and data (L1d) caches as well as an L2 cache. The L3 and L4 caches, along with the main memory, are shared among all cores. A graphical representation of the memory hierarchy is displayed in Fig. 1. To obtain such a figure, we ask `hwloc-ls` to output in Portable Document Format (PDF) format

```
[user@eu-login-39 ~]$ hwloc-ls --whole-system --no-io --of pdf XEON_E3-1284Lv4.pdf
```

Note that the PDF contains the topology in vector graphics format scalability without loss of resolution, which is standard in scientific publications. For more information on the used commands and their options, please have a look at the man pages (e.g., `man lscpu`).

Next, we summarize the collected information in the same style that you should report your results for a node of the Euler VII (phase 1 and 2) cluster.

The memory hierarchy of an Euler login node with an Intel Xeon E3-1284Lv4 quad-core CPU is summarized in Table 1. The memory topology is displayed in Fig. 1 and each core has its own L1 and L2 cache, while L3 and L4 caches are shared among all the four cores.

Main memory	32 GB
L4 cache	128 MB
L3 cache	6 MB
L2 cache	256 KB
L1 cache	32 KB

Table 1: Memory hierarchy of an Euler login node with an Intel Xeon E3-1284Lv4 quad-core CPU. Each core has its own L1 and L2 cache, while L3 and L4 caches are shared among all the four cores.

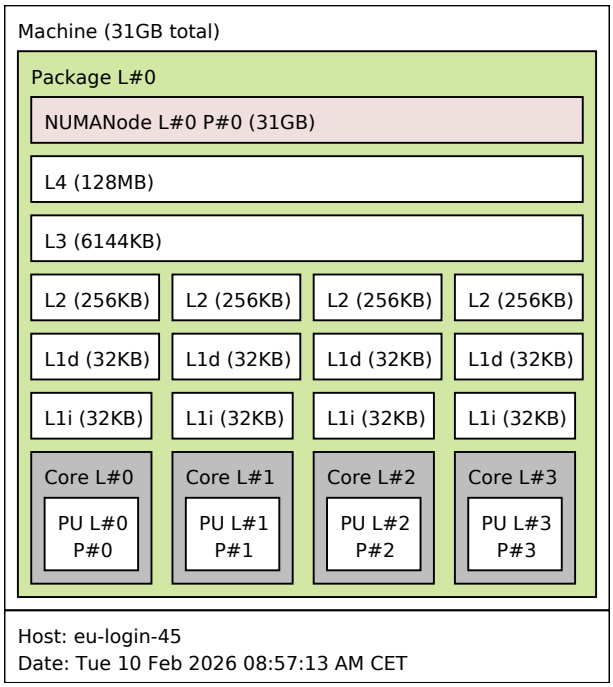


Figure 1: Topology of Euler login node with an Intel Xeon E3-1284Lv4 quad-core CPU.

2.3 Bandwidth: STREAM benchmark [10 Points]

Now that we have an overview of the peak performance, as well as the topology and size of the memory system, another key performance characteristic to consider is the speed of the memory system. The speed of the memory system is quantitatively measured in terms of its bandwidth, which indicates the amount of data that can be transferred within a specific time frame. McCalpin's STREAM benchmark [8] is a widely recognized tool used for measuring the memory bandwidth of a CPU. It consists of four operations or kernels — Copy, Scale, Add, and Triad — that evaluate the sustainable memory bandwidth and the corresponding computation rate for simple vector kernels (see, e.g, [3, Sec. 3.1.2]).

In this task, we ask you to run the STREAM benchmark on the Euler VII (phase 1 and 2) nodes in order to measure the *single-core* bandwidth. Find below a step-by-step guide:

1. Download the STREAM Benchmark: it can be obtained from the official website[11] in the source code directory[12]. You will need `stream.c` and `mysecond.c`.

[11] https://www.cs.virginia.edu/stream/
[12] https://www.cs.virginia.edu/stream/FTP/Code/

2. In `stream.c`, you will find precise instructions on how to compile and run the benchmark. It is particularly important to tune the sizes of the arrays (used in the Copy, Scale, Add, and Triad kernels) to match the cache sizes of the system of interest: Each array must be at least four times the size of the last cache level. This is set by the `STREAM_ARRAY_SIZE` preprocessor macro and two examples are given in the `stream.c`. For the CPU studied in the example of Section 2.2.1, we would then

```
module load stack gcc # load a stack and a compiler
gcc -O3 -march=native -DSTREAM_TYPE=double -DSTREAM_ARRAY_SIZE=68000000 \
    -DNTIMES=20 stream.c -o stream_c.exe # compile
sbatch --mem-per-cpu=2G --wrap "./stream_c.exe" # submit to queue and run
                                                # (require enough memory!)
```

   Note that you want to adapt the target architecture (`-march` option) accordingly or write a batch script that compiles directly on the compute node. Likewise, the array sizes need to be to the target architecture according to the manual.

3. The output (`slurm-jobid.out`) produced:

```
-------------------------------------------------------------
STREAM version $Revision: 5.10 $
-------------------------------------------------------------
This system uses 8 bytes per array element.
-------------------------------------------------------------
Array size = 68000000 (elements), Offset = 0 (elements)
Memory per array = 518.8 MiB (= 0.5 GiB).
Total memory required = 1556.4 MiB (= 1.5 GiB).
Each kernel will be executed 20 times.
 The *best* time for each kernel (excluding the first iteration)
 will be used to compute the reported bandwidth.
-------------------------------------------------------------
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 61118 microseconds.
   (= 61118 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-------------------------------------------------------------
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           19934.4     0.055412     0.054579     0.056215
Scale:          12020.9     0.092785     0.090509     0.094412
Add:            12872.5     0.129225     0.126782     0.131574
Triad:          12780.8     0.129039     0.127692     0.130467
-------------------------------------------------------------
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-------------------------------------------------------------
```

   We observe that the bandwidths resulting from the Scale, Add, and Triad kernels are roughly consistent, while the bandwidth for the Copy kernel appears to be substantially higher. There may be several explanations for this discrepancy, but they are beyond the scope of this project (and course). For a rough estimate, we can assume a maximum bandwidth $b_{\text{STREAM}} = 12$ GB/s.

Of course, you will need to adapt some these steps to the task. In your report, follow a similar style to the above description. Include source code (including build files), batch scripts, and Slurm output files in your submission.

For further context and advice regarding the STREAM benchmark, we recommend looking at the official website.

2.4 Performance model: A simple roofline model [16 Points]

The *roofline* model is a visual representation that serves as a powerful tool for evaluating the performance potential of computing systems, especially in the context of high-performance computing [11]. It plots achievable performance (in GFlops/s) against operational intensity (in Flops/Byte), delineating the maximum performance given by the system's peak performance and the memory bandwidth constraints. The "roofline" itself consists of two main segments: the *memory-bound* region, where performance is limited by the system's memory bandwidth, and the *compute-bound* region, where performance is capped by the peak computational power of the processor. The frontier between both regimes is called the *ridge point*. This model allows developers and researchers to identify whether their algorithms are memory-bound or compute-bound and to understand the performance implications of hardware limitations, guiding optimizations towards achieving maximum efficiency within the given hardware constraints.

A simple (or sometimes called naive) roofline model for a hypothetical system combining Sections 2.1 and 2.2.1 is displayed in Fig. 2. We observe that the ridge point is roughly at an operational intensity of $I_{\text{ridge}} \approx 4$. Hence, a given kernel or application with an operational intensity below I_{ridge} is memory-bound. Similarly, a given kernel or application with an operational intensity above I_{ridge} is compute-bound.

Your tasks are:

1. With your performance data collected in the previous tasks, create a roofline model for both single cores of the Euler VII (phase 1 and 2) nodes.
2. Visualize your roofline models in a plot similar to Fig. 2.
3. At what operational intensity is a kernel or application memory/compute-bound?

All plots must be created using scriptable tools (Python, gnuplot, etc.) — spreadsheet software is not accepted — and you must submit all used plotting scripts.

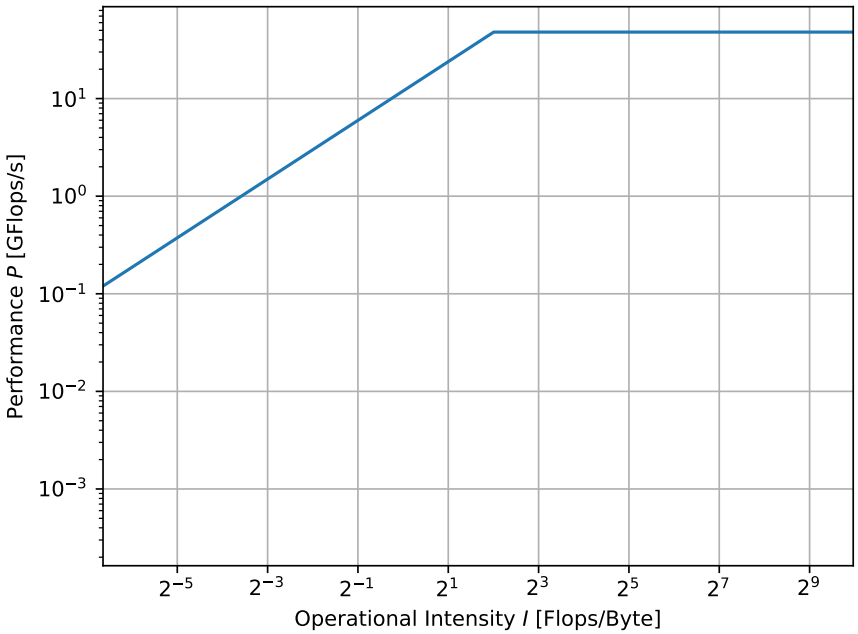


Figure 2: Simple (or naive) roofline model for a single core of a hypothetical CPU.

3 Auto-vectorization [10 points]

Read the Chapter "Automatic Vectorization" of the *Intel oneAPI DPC++/C++ Compiler Developer Guide and Reference* [5] and answer briefly the following questions:

1. What is data alignment and is it important for data structures to be aligned?

2. What are some obstacles that can prevent automatic vectorization by the compiler?
3. Is there a way to help the compiler to vectorize and how?
4. Which loop optimizations are performed by the compiler to vectorize and pipeline loops?
5. What can be done if automatic vectorization by the compiler fails or is sub-optimal?

If you wish, you can play around with (a slightly outdated version of) the Intel compiler by loading the following module:

```
[user@eu-login-39 ~]$ module load stack oneapi
[user@eu-login-39 ~]$ icx --version
Intel(R) oneAPI DPC++/C++ Compiler 2023.2.0 (2023.2.0.20230622)
Target: x86_64-unknown-linux-gnu
Thread model: posix
InstalledDir: ...
```

4 Matrix multiplication optimization[13] [30 points]

4.1 Motivation

Linear algebra is a cornerstone operation in computational science and holds a particularly pivotal role in HPC. In particular, we are going to focus on matrix multiplication. This mathematical operation underpins a vast array of scientific and engineering disciplines, facilitating the modeling of complex systems, simulations of physical phenomena, and the processing of large datasets. In HPC environments, the efficiency and scalability of matrix multiplication directly influence the performance of algorithms (used in computational sciences, engineering, machine learning, and beyond). This functionality is offered by BLAS (Basic Linear Algebra Subprograms), a specification encompassing a suite of low-level routines designed for executing common linear algebra operations efficiently.

Each manufacturer typically supplies a BLAS library (e.g., Intel's MKL library, AMD's AOCL, ...), carefully hand-optimized for its machines. It includes a large number of routines, not just matrix multiplication, although matrix multiplication is one of the most important, because it is used as a benchmark to compare the speed of computers (e.g., LINPACK Benchmark[14] used in the Top 500[15] list of the fastest supercomputers). In addition to manufacturer supplied libraries, there are also a number of open source projects. For instance, there are the ATLAS[16], GotoBLAS[17], OpenBLAS[18], and BLIS[19] (the 2023 recipient of the James H. Wilkinson Prize for Numerical Software). The list is not meant to be exhaustive.

The performance difference between naive and library implementations can be drastic. Figure 3 shows the performance in GFlops/s of n-by-n matrix multiplication (on a single core of an AMD EPYC 7763 CPU) as a function of matrix size n. It compares the highly optimized Intel MKL and OpenBLAS libraries with a naive implementation. As apparent from the figure, the naive implementation is sub-optimal (to say the least). As it turns out, matrix multiplication, commonly referred to as `DGEMM` (Double precision GEneral Matrix Multiplication), has a growing operational intensity (with the matrix size n). Therefore it can be (asymptotically) pushed into the compute bound regime (e.g., roofline model). However, achieving this requires very specialized, hardware dependent, optimization techniques. It is the goal of this project task to familiarize you with some basic techniques and provide you some pointers to the literature for more advanced techniques in the next paragraph.

The anatomy of a HPC matrix multiplication is described in the article [2]. The BLIS framework is described by Zee et al. [12] and Low et al. [6] propose an analytical performance model to determine certain parameters. The latter article gives a nice overview of the anatomy of a fast `DGEMM`. The recent article by Alaejos et al. [1] provides templates to write micro-kernels using vector intrinsics. Last but not least, we highly recommend the very comprehensive online course "LAFF-On Programming for High Performance" [10]. On a less scientific level, you might also find the newspaper article [7] interesting.

[13] This task is based on a previous HPC Lab for CSE by Prof. Olaf Schenk from the Institute of Computing at the Faculty of Informatics at Universita della Svizzera Italiana (USI), which was itself originally based on a tutorial from Prof. Katherine A. Yelick from the Computer Science Department at the University of Berkeley (http://www.cs.berkeley.edu/~yelick/).
[14] https://www.top500.org/project/linpack/
[15] https://www.top500.org/
[16] https://math-atlas.sourceforge.net/
[17] https://en.wikipedia.org/wiki/GotoBLAS
[18] https://www.openblas.net/
[19] https://github.com/flame/blis

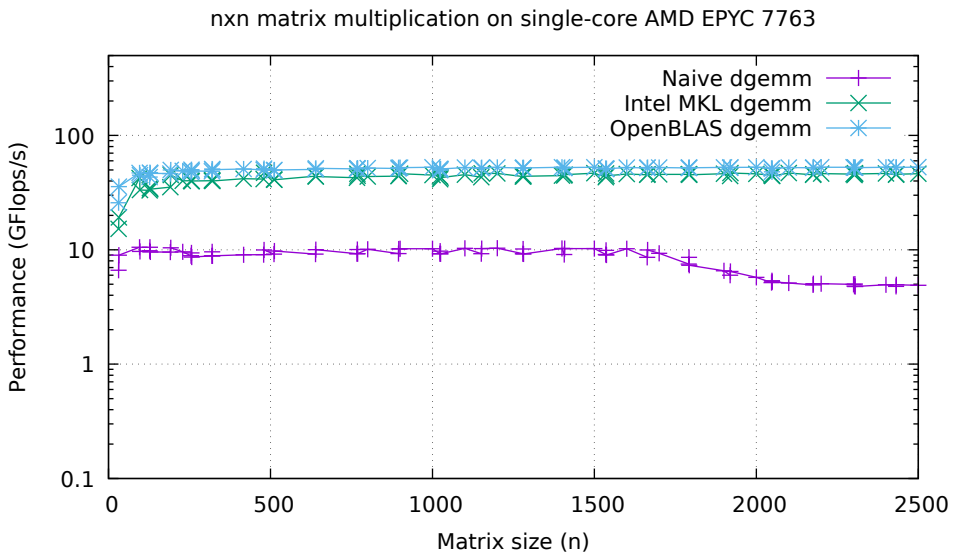


Figure 3: Serial performance of matrix multiplication on the AMD EPYC 7763.

4.2 Importance of data access optimization

Among the various factors that can limit performance in HPC, data access stands out as the most critical. This is due to the inherent imbalance in modern CPUs, where there is a significant discrepancy between their theoretical peak performance and the available memory bandwidth. Therefore, any attempt at optimization should initially focus on minimizing data traffic, or, if that proves to be impractical, strive to make data transfer as efficient as possible. If you haven't already, we strongly recommend to read Chapter 3 of the book [3].

4.2.1 Modeling memory hierarchies

Real memory hierarchies are very complicated, so modeling them carefully enough to predict the performance of an algorithm is hard. To make progress, we will use the following simplified model:

(i) There are only **two levels** in the memory hierarchy, **fast** (e.g., registers, caches) and **slow** (e.g., main memory).

(ii) All the input data starts in the slow level, and the output must eventually be written back to the slow level.

(iii) The size of the fast memory is M, which is much smaller than the slow memory. In particular, the input of large problems will not fit in the fast memory simultaneously.

(iv) The programmer explicitly controls which data moves between the two memory levels and when. **Remark**: It might seem incorrect to suggest that programmers can control data movement between main memory and cache, as this process is typically automated by the hardware. However, we know how the hardware works: it moves data to the cache precisely when the user first tries to load it into a register to perform arithmetic, and puts it back in main memory when the cache is too full to get the next requested word. This means that we can write programs as if we controlled the cache explicitly by doing arithmetic operations in different orders. Thus, two programs that do the same arithmetic in different orders may run at very different speeds, because one reduces data movement between the main memory and cache.

(v) Arithmetic (and logic) can only be done on data residing in the fast memory and data movement and computation cannot overlap. Each arithmetic operation takes time t_a. Moving a word of data from one memory to the other takes time $t_m \gg t_a$. Hence, if a program performs m memory moves and a arithmetic operations, the total time it takes is $T = a \cdot t_a + m \cdot t_m$, where it may be that $m \cdot t_m \gg a \cdot t_a$.

With this model, let us get a **lower bound** on the speed of any algorithm for a problem that has m_i inputs and m_o outputs, and does a arithmetic operations. The only difference between algorithms that

we permit is the order in which the arithmetic operations are executed. According to our model, the run-time taken will be at least

$$T = a \cdot t_a + (m_i + m_o) \cdot t_m,$$

no matter which clever order we do the arithmetic in.

For example, suppose the problem is to take two input arrays of n numbers each, $a[i]$ and $b[i]$ for $i = 1, .., n$, and produce two output arrays s and p, where $s[i] = a[i] + b[i]$ and $p[i] = a[i] \cdot b[i]$. Then the number of arithmetic operations is $a = 2n$, and the number of memory moves is $m_i = 2n$ and $m_o = 2n$. Thus, a lower bound on the run-time for any algorithm for this problem is $T = 2nt_a + 4nt_m$. Let us look at two different algorithms for this problem, and analyze which will be faster. The two algorithms are listed in Table 2. According to our model, we assume that a and b are initially in the slow memory, and $M \ll n$, so a and b cannot fit in the fast memory, and we have indicated when loads from and stores to slow memory occur. The point is that Algorithm 1 loads a and b twice, whereas Algorithm 2 only loads

Algorithm 1	Algorithm 2

`for i = 1, n`
`  Load a[i], b[i] into fast memory`
`  s[i] = a[i] + b[i]`
`  Store s[i] into slow memory`
`end for`
 	
`for i = 1, n`
`  Load a[i], b[i] into fast memory`
`  p[i] = a[i] * b[i]`
`  Store p[i] into slow memory`
`end for`
	`for i = 1, n`
	`  Load a[i], b[i] into fast memory`
	`  s[i] = a[i] + b[i]`
	`  Store s[i] into slow memory`
	`  p[i] = a[i] * b[i]`
	`  Store p[i] into slow memory`
	`end for`

Table 2: Two algorithms to compute the sum and product of the arrays.

them once (remember, there is not enough room in the fast memory to keep all the $a[i]$ and $b[i]$ for a second pass to compute $p[i]$). As per our simple model, Algorithm 1 takes run-time $T_1 = 2nt_a + 6nt_m$, whereas Algorithm 2 takes only $T_2 = 2nt_a + 4nt_m$, the minimum possible. Thus, for $t_a \ll t_m$, Algorithm 1 takes 50% longer to run than Algorithm 2. In the case of matrix multiplication, we will observe an even more dramatic difference.

4.2.2 Minimizing slow memory access in matrix multiplication

We will only consider different ways to implement n-by-n matrix multiplication $C = C + A \cdot B$ using $2n^3$ operations (i.e., we will not consider Strassen's method [9] and variants thereof). Thus, the only difference between algorithms will depend on the number of loads and stores to the slow memory. We also assume that the slow memory is large enough to contain our three n-by-n matrices A, B and C, but the fast memory is too small for this. Otherwise, if the fast memory were large enough to contain A, B, and C simultaneously, then our algorithm would simply be:

```
1 Load A, B and C from slow into fast memory
2
3 method dgemm_fastmem(A, B, C)
4
5   Compute C = C + A · B entirely in fast memory
6
7 end method dgemm_fastmem
8
9 Store the result C back to slow memory
```

The number of slow memory accesses for this algorithm is $4n^2$ ($m_i = 3n^2$ loads of A, B, and C into fast memory, and $m_o = n^2$ stores of C to slow memory), yielding a run-time of $T = 2n^3 t_a + 4n^2 t_m$. Clearly, no algorithm doing $2n^3$ arithmetic operations can be faster. At the extreme where the fast memory is very small ($M = 1$), then there will be at least 1 memory reference per operand for each arithmetic operation involving entries of A and B, for a run-time of at least $T = 2n^3 t_a + (2n^3 + 2n^2)t_m$.

So, when the size of fast memory M satisfies $1 \ll M \ll 3n^2$, what is the fastest algorithm?

This is the practical question for large matrices, since real caches have thousands of entries. As we just saw, the worst case run-time is $2n^3t_a + (2n^3 + 2n^2)t_m$, and the best we can hope for is $2n^3t_a + 4n^2t_m$, which would be almost $n/2$ times faster for $t_m \gg t_a$.

We begin by analyzing the simplest matrix multiplication algorithm, which we repeat below, including descriptions of when data moves between the slow and the fast memories. Remember that A, B, and C all start in the slow memory, and that the result C must be finally stored in the slow memory.

```
1  Naive matrix multiplication C = C + A · B
2
3  method dgemm_naive(A, B, C)
4
5    for i = 1, n
6      for j = 1, n
7        Load c_{i,j} into fast memory
8        for k = 1, n
9          Load a_{i,k} into fast memory
10         Load b_{k,j} into fast memory
11         c_{i,j} = c_{i,j} + a_{i,k} * b_{k,j}
12       end for
13       Store c_{i,j} into slow memory
14     end for
15   end for
16
17 end method dgemm_naive
```

Let m_{naive} denote the number of **slow memory references** in this naive algorithm. Then

$$\begin{aligned} m_{\text{naive}} &= n^3 && \dots \text{ for loading each entry of A } n \text{ times} \\ &+ n^3 && \dots \text{ for loading each entry of B } n \text{ times} \\ &+ 2n^2 && \dots \text{ for loading and storing each entry of C once} \\ &= 2n^3 + 2n^2, \end{aligned}$$

or about as many slow memory references as arithmetic operations. Thus, the run-time is $T_{\text{naive}} = 2n^3t_a + (2n^3 + 2n^2)t_m$, the worst possible.

Let us analyse an improved algorithm called **blocked matrix multiplication** (sometimes it is called **tiled** or **panelled** instead of **blocked**). The n-by-n matrix A is divided into smaller s-by-s sub-matrices or blocks A_{ij}, where s is a parameter called the block size that we will specify later. We assume s divides n for simplicity. Matrices B and C are similarly partitioned. Then, we can think of A as an n/s-by-n/s block matrix, where each entry $A_{i,j}$ is an s-by-s block. The inner loop $C_{i,j} = C_{i,j} + A_{i,k} \cdot B_{k,j}$ now runs for $k = 1$ to s, and represents an s-by-s matrix multiplication and addition. The algorithm becomes:

```
1  Blocked matrix multiplication C = C + A · B
2
3  method dgemm_blocked(A, B, C)
4
5    for i = 1, n/s
6      for j = 1, n/s
7        Load C_{i,j} block into fast memory
8        for k = 1, n/s
9          Load A_{i,k} block into fast memory
10         Load B_{k,j} block into fast memory
11         dgemm_fastmem(A_{i,k}, B_{k,j}, C_{i,j})
12       end for
13       Store C_{i,j} into slow memory
14     end for
15   end for
16
17 en method dgemm_blocked
```

The inner loop `dgemm_fastmem`$(A_{i,k}, B_{k,j}, C_{i,j})$ has all its data residing in the fast memory, and so causes no slow memory traffic at all. Redoing the count of slow memory references yields

$$\begin{aligned} m_{\text{blocked}} = m_{\text{blocked}}(s) &= (n/s)^3 \cdot s^2 && \dots \text{ for loading each block } A_{i,k} \ (n/s)^3 \text{ times} \\ &+ (n/s)^3 \cdot s^2 && \dots \text{ for loading each block } B_{k,j} \ (n/s)^3 \text{ times} \\ &+ 2(n/s)^2 \cdot s^2 && \dots \text{ for loading and storing each block } C_{i,j} \text{ once} \\ &= 2n^3/s + 2n^2. \end{aligned}$$

Comparing $m_{\text{naive}} = 2n^3 + 2n^2$ and $m_{\text{blocked}} = 2n^3/s + 2n^2$, it is obvious that we want to pick s as large as possible to make $m_{\text{blocked}}(s)$ as small as possible. But how big can we pick s? The largest possible value is obviously $s = n$, which corresponds to loading all of A, B and C into the fast memory, which we cannot do. So, s depends on the size M of the fast memory, and the constraint it must satisfy is that the three s-by-s blocks $A_{i,k}, B_{k,j}$, and $C_{i,j}$ must simultaneously fit in the fast memory, which implies $3s^2 \leq M \implies s \leq \sqrt{M/3}$. Therefore, the largest value $s = \sqrt{M/3}$ yields

$$m_{\text{blocked}}(\sqrt{M/3}) = \sqrt{12}\frac{n^3}{\sqrt{M}} + 2n^2.$$

In other words, for large matrices (large n) we decrease the number of slow memory references, the most expensive operation, by a factor $\mathcal{O}(M)$. This is attractive, because it says that cache (fast memory) helps, and the larger the cache the better.

In summary, the running time for this algorithm is

$$T_{\text{blocked}} = 2n^3 \cdot t_a + (\sqrt{12}\frac{n^3}{\sqrt{M}} \cdot t_m).$$

There is a theorem, which we will not prove, that says that up to constant factors, we cannot do fewer slow memory references than this (while doing the usual $2n^3$ arithmetic operations):

Theorem (Hong + Kung, 1981, 13th Symposium on the Theory of Computing [4]): Any implementation of matrix multiplication using $2n^3$ arithmetic operations performs at least $\mathcal{O}(n^3/\sqrt{M})$ slow memory references.

In practice, this technique is very important to get matrix multiplication to run as fast as possible. But careful attention must also be paid to other details of the instruction set, arithmetic units, and so on. If there are more levels of memory hierarchy (two levels of cache), then one might use this technique recursively, dividing s-by-s blocks into yet smaller blocks to exploit the next level of memory hierarchy. However, this goes beyond the scope of this project and course.

4.3 Optimizing square matrix multiplication

Your task is now to write a blocked matrix multiplication (as outlined above) and optimize it for a single core of the AMD EPYC 7763 on Euler.

Skeleton code

Download the skeleton code provided on the course webpage page. It contains the following:

- `dgemm-naive.c` – For illustrative purposes, a naive implementation of matrix multiplication using three nested loops.
- `dgemm-blas.c` – A wrapper which calls an optimized BLAS implementation of matrix multiplication (OpenBLAS).
- `dgemm-blocked.c` – A skeleton for a blocked implementation of matrix multiplication. It is your task to implement and optimize the `square_dgemm` function in this file.
- `benchmark.c` – A driver program that generates matrices of a number of different sizes and benchmarks the performance. It outputs the performance in GFlops/s and in a percentage of theoretical peak performance attained. During development, you may find it useful to reduce the tested matrix sizes to decrease total run time. If testing on a different computer, you may also need to update the peak performance constant `MAX_SPEED`.

- `Makefile` – A simple makefile to build the executables for the benchmarking of the naive, optimized BLAS and your blocked implementation.
- `run_matrixmult.sh` – A job script that executes all three executables and produces log files (`*.data`) that contain the performance logs. It also plots the data in the performance logs and produces `timing.pdf` showing the results.

Familiarize yourself with the code and the used conventions. In particular, note that we use the *column major order* storage scheme to conform with BLAS' Fortran origins.

Running the code

The skeleton code should run out of the box and a file listing should look as:

```
[user@eu-login-39]$ ls
benchmark.c   dgemm-blocked.c  Makefile           timing.gp
dgemm-blas.c  dgemm-naive.c    run_matrixmult.sh
```

We will use the GNU Compiler Collection and the OpenBLAS implementation, which can be loaded on Euler as follows:

```
[user@eu-login-39]$ module load stack gcc openblas
```

Building the code

```
[user@eu-login-39]$ make
```

generates the four executables `benchmark-naive`, `benchmark-jki` `benchmark-blas` and `benchmark-blocked`. We submit to the batch system with

```
[user@eu-login-39]$ sbatch run_matrixmult.sh
Submitted batch job 49100417
[user@eu-login-39]$ squeue
     JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
  49100417 normal.4h matrixmu     user  R       0:08      1 eu-a2p-532
```

When our job is finished, you will find new files in the directory containing the output of the benchmark programs. For example, we will find the files `matrixmult-jobid.out` and `matrixmult-jobid.err`. The first file contains the standard output and the second file contains the standard error. Additionally, the performance data are stored in `*.data` files and `timing.pdf` is a plot of the performance.
Note: The benchmark of the provided naive implementation(s) may take a long time to run. Consider disabling it during development.

To view the generated PDF plot, you can copy it on your machine or use **ImageMagick's** `display` command. First, ensure you have the necessary module loaded:

```
[user@eu-login-39]$ module load stack ghostscript imagemagick
```

Then, use the following command to open the plot:

```
[user@eu-login-39]$ display plot.pdf
```

Important: You must be logged in with **X forwarding enabled** for this to work. When connecting via SSH, use the `-X` or `-Y` flag:

```
ssh -X username@cluster
```

or

```
ssh -Y username@cluster
```

Optimizing square matrix multiplication [30 points]

- **Implementation**:
  - Improve the loop ordering in `dgemm_jki.c`.
  - Implement blocking for square matrix multiplication in `dgemm_blocked.c`.

- **Optimization**: Optimize your code to maximize its performance. Consider various strategies, such as compiler options and tuning data access patterns (including block size adjustments).
- **Documentation and Analysis**: Document the used or attempted optimizations in your report, including performance graphs and, if applicable, tables. Provide a detailed description of your experimental setup. Compare your optimized implementation to the OpenBLAS library.
- **[Bonus 20 points]**: Employ any advanced techniques from library implementations as described in the references provided in the motivation Section 4.1, including any references therein or other relevant resource. However, remember to accurately cite all your sources.

A Additional notes and submission details

Submit your **report** and **all source code files**, including your build files (e.g., `Makefile`(s)) and other scripts (e.g., batch job scripts, plotting scripts), as a compressed tar archive. Your LaTeX report must be based on the provided template from the course webpage and should summarize your results and observations for all sections. Your submission should be uploaded to SAM-UP and must include

- Your report, named as `project_number_lastname_firstname.pdf`. You may discuss all questions with anyone you like. However, (i) you must list all collaborators in your submission, and (ii) you must prepare and write up the submission independently.
- A compressed tar[20] archive, named `project_number_lastname_firstname.tar.gz`, containing:
  - All the source codes of your solutions.
  - Build files and scripts. If you have modified the provided build files or scripts, ensure they still build the sources and run correctly. We will use them to grade your submission.
- The total size of your submission (report PDF and archive) must not exceed 25 MB.
- File names must contain only `ASCII` characters (avoid spaces, special symbols, and non-English letters).

Please strictly follow these instructions and naming conventions. Failure to do so results in additional work for the teaching assistants.

References

[1] Guillermo Alaejos, Adrián Castelló, Héctor Martínez, Pedro Alonso-Jordá, Francisco D. Igual, and Enrique S. Quintana-Ortí. Micro-kernels for portable and efficient matrix multiplication in deep learning. *The Journal of Supercomputing*, 79(7):8124–8147, May 2023. doi:10.1007/s11227-022-05003-3.

[2] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, May 2008. doi:10.1145/1356052.1356053.

[3] Georg Hager and Gerhard Wellein. Introduction to high performance computing for scientists and engineers. *Chapman & Hall/CRC Computational Science*, July 2010. URL: http://dx.doi.org/10.1201/EBK1439811924, doi:10.1201/ebk1439811924.

[4] Jia-Wei Hong and H. T. Kung. I/O complexity: The red-blue pebble game. In *Proceedings of the thirteenth annual ACM symposium on Theory of computing*, STOC '81, pages 326–333, New York, NY, USA, May 1981. Association for Computing Machinery. URL: https://dl.acm.org/doi/10.1145/800076.802486, doi:10.1145/800076.802486.

[5] Intel. Intel oneAPI DPC++/C++ Compiler Developer Guide and Reference. https://cdrdv2.intel.com/v1/dl/getContent/834245?fileName=dpcpp-cpp-compiler_developer-guide-reference_2025.0-767253-834245.pdf, 2025. [Online; accessed 12-February-2025].

[20] https://en.wikipedia.org/wiki/Tar_(computing)

[6] Tze Meng Low, Francisco D. Igual, Tyler M. Smith, and Enrique S. Quintana-Orti. Analytical Modeling Is Enough for High-Performance BLIS. *ACM Transactions on Mathematical Software*, 43(2):12:1–12:18, August 2016. URL: https://dl.acm.org/doi/10.1145/2925987, doi:10.1145/2925987.

[7] John Markoff. Writing the Fastest Code, by Hand, for Fun: A Human Computer Keeps Speeding Up Chips. *The New York Times*, 2005. [Online; accessed 24-February-2024]. URL: https://www.nytimes.com/2005/11/28/technology/writing-the-fastest-code-by-hand-for-fun-a-human-computer-keeps.html?smid=url-share.

[8] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE computer society technical committee on computer architecture (TCCA) newsletter*, 2, December 1995. URL: https://www.cs.virginia.edu/~mccalpin/papers/balance/.

[9] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, aug 1969. doi:10.1007/bf02165411.

[10] Robert van de Geijn, Margaret Myers, and Devangi Parikh. LAFF - On Programming for High Performance. https://www.cs.utexas.edu/users/flame/laff/pfhp/, November 2021. [Online; accessed 24-February-2024].

[11] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65, April 2009. doi:10.1145/1498765.1498785.

[12] Field G. Van Zee, Tyler M. Smith, Bryan Marker, Tze Meng Low, Robert A. Van De Geijn, Francisco D. Igual, Mikhail Smelyanskiy, Xianyi Zhang, Michael Kistler, Vernon Austel, John A. Gunnels, and Lee Killough. The BLIS framework. *ACM Transactions on Mathematical Software*, 42(2):1–19, jun 2016. doi:10.1145/2755561.